

Tema 3: Ficheros y Llamadas al Sistema

En Sistemas Operativos, un fichero es mucho mas que un archivo en disco. Es una abstraccion proporcionada por el sistema operativo que representa datos persistentes, un dispositivo, almacenamiento volatil, o cualquier recurso en general. Las operaciones sobre ficheros son llamadas al sistema, y entenderlas bien es fundamental para el examen.

Descriptores de fichero

Un descriptor de fichero es un numero entero que identifica un fichero abierto para un proceso. Es el indice en la tabla de descriptores de fichero del proceso, que esta en el kernel. Cuando abres un fichero, el sistema operativo te devuelve un descriptor que usaras para todas las operaciones posteriores.

Hay tres descriptores estandar que todo proceso tiene abiertos desde el principio: el cero es stdin, la entrada estandar; el uno es stdout, la salida estandar; y el dos es stderr, la salida de errores. Estos tres numeros los debes memorizar porque aparecen en muchas preguntas de examen.

La llamada open

La llamada open abre un fichero y devuelve un descriptor. Recibe como minimo dos parametros: la ruta del fichero y las banderas que indican el modo de apertura. Si incluye la bandera O_CREAT, necesitas un tercer parametro con los permisos.

Las banderas principales son: O_RDONLY para solo lectura, O_WRONLY para solo escritura, O_RDWR para lectura y escritura, O_CREAT para crear el fichero si no existe, O_TRUNC para truncar el fichero a longitud cero si ya existe, O_APPEND para escribir siempre al final, y O_CLOEXEC para cerrar automaticamente el descriptor al hacer exec.

Open devuelve un numero mayor o igual a cero si tiene exito, o menos uno si hay error. Un dato importantisimo para el examen: si abres el mismo fichero dos veces con open, obtienes dos descriptores diferentes con offsets independientes. No comparten la posicion de lectura ni de escritura.

La mascara umask

Umask es una mascara de creacion de ficheros que es propiedad del proceso y se hereda del padre. Funciona asi: los bits que estan a uno en la mascara se excluyen de los permisos del fichero creado. La formula es: permisos finales igual a modo AND NOT umask. Por ejemplo, si umask es cero setenta y siete y creas un fichero con modo seiscientos sesenta y seis, los permisos finales seran seiscientos, que es lectura y escritura solo para el propietario.

La llamada read

Read lee bytes de un fichero abierto. Recibe tres parametros: el descriptor de fichero, un buffer donde almacenar los datos leidos, y el numero maximo de bytes a leer. Devuelve el

numero de bytes realmente leidos, que puede ser menor que el solicitado sin que eso sea un error. Si devuelve cero, significa fin de fichero. Si devuelve menos uno, hay un error.

Esto es pregunta de examen segura: read puede devolver menos bytes de los pedidos y eso no es un error. Se llama lectura corta, y es completamente normal. Tambien es importante saber que read es una llamada bloqueante: si no hay datos disponibles, el proceso se queda esperando hasta que los haya.

La llamada write

Write escribe bytes en un fichero abierto. Recibe el descriptor, un buffer con los datos, y el numero de bytes a escribir. Devuelve el numero de bytes escritos, o menos uno en caso de error. Al igual que read, write puede escribir menos bytes de los pedidos sin que sea error, y deberias comprobarlo.

Pregunta clasica de examen: tienes un write y compruebas si el valor de retorno es menor que cero. La respuesta tipica es que esa comprobacion no es suficiente, porque write podria devolver un numero menor que el solicitado sin ser menos uno, y eso tambien deberia manejarse.

La llamada lseek

Lseek cambia la posicion de lectura y escritura dentro de un fichero abierto. Recibe el descriptor, un offset, y una referencia que puede ser SEEK SET para posicion absoluta desde el principio, SEEK CUR para posicion relativa desde la actual, o SEEK END para posicion relativa desde el final. Devuelve la nueva posicion o menos uno si hay error.

Tambien existen pread y pwrite, que son versiones de read y write que leen y escriben en una posicion especifica sin modificar el offset del descriptor. Son utiles para acceso concurrente.

La llamada close

Close cierra un fichero y libera el descriptor. Devuelve menos uno si hay error. Es importante cerrar los ficheros cuando ya no los necesitas para no desperdiciar descriptores.

Las llamadas stat y access

La llamada stat lee los metadatos de un fichero en una estructura. Los campos mas importantes son: st ino, que es el numero de i-nodo del fichero; st mode, que contiene los permisos y el tipo; st nlinks, que es el numero de enlaces duros; st uid y st gid, que son el propietario y el grupo; st size, que es el tamaño en bytes; y los tiempos de acceso, modificacion y cambio de metadatos.

Una nota importante: stat atraviesa enlaces simbolicos y te da informacion del fichero destino. Si quieres informacion del enlace simbolico en si, usa lstat.

Access comprueba si se puede acceder a un fichero con ciertos permisos: F OK para comprobar existencia, R OK para lectura, W OK para escritura, y X OK para ejecucion.

Las llamadas dup y dup2

Dup duplica un descriptor de fichero en el primer slot libre de la tabla de descriptors. Dup2 duplica en un slot específico, cerrando primero el que había ahí si estaba abierto. Ambos descriptors, el original y el duplicado, comparten el offset.

Estas llamadas son fundamentales para implementar redirecciones en el shell y pipes entre procesos. Cuando el shell ejecuta un comando con redirección, usa dup2 para sustituir stdin o stdout por el fichero deseado antes de hacer exec.

Un ejemplo típico de examen: abres un fichero en modo lectura, haces dup2 para poner ese descriptor en la posición cero, que es stdin, cierras el descriptor original, y luego haces `execl` de `wc` menos `l`. El resultado es que `wc` lee del fichero en vez de del terminal, y escribe el número de líneas por stdout.

La llamada unlink

Unlink elimina un nombre, es decir, un enlace duro, de un fichero. Si es el último nombre y no hay procesos con el fichero abierto, el fichero se borra realmente. Pero si quedan otros enlaces duros apuntando al mismo i-nodo, el fichero sigue existiendo.

Ejemplo de examen: creas un fichero `a`, luego haces `ln a b` para crear un enlace duro. El i-nodo tiene `st_nlink` igual a dos. Si luego haces `unlink` de `a`, `st_nlink` baja a uno. El fichero sigue existiendo porque `b` aun lo referencia. Solo cuando `st_nlink` llega a cero y ningún proceso tiene el fichero abierto, se libera realmente.

Buffering: fopen, fprintf y fread vs open, read, write

Hay dos familias de funciones para ficheros. Las llamadas al sistema como `open`, `read`, `write` y `close` son directas al kernel. Las funciones de la biblioteca estándar como `fopen`, `fread`, `fprintf` y `fclose` añaden un buffer en espacio de usuario.

La ventaja del buffering es que se hacen menos llamadas al sistema. Si usas `fread` para leer de treinta y dos en treinta y dos bytes, internamente la biblioteca hace una llamada al sistema con un bloque grande y te va sirviendo los datos del buffer. Esto es más eficiente.

Pero el buffering puede ser una trampa. Ejemplo clásico de examen: haces `fopen` de un fichero en modo escritura, luego `fprintf` para escribir, y después un bucle infinito. La pregunta es si se escribe algo en el fichero. La respuesta es que probablemente no, porque los datos están en el buffer en memoria y el programa nunca llega a hacer flush ni a cerrar el fichero. El contenido nunca llega al disco.